

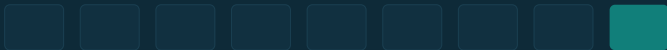


STREAM PROCESSING AVEC APACHE KAFKA

Traitement avec état

Agrégations et state stores dans Kafka Streams

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs

- **Comprendre pourquoi l'état est nécessaire**

Compter, cumuler, agréger : autant d'opérations qui ont besoin de mémoire.

- **Grouper puis agréger**

`groupByKey / groupBy`, puis `count`, `reduce`, `aggregate` → un `KTable`.

- **Connaître les state stores**

Un store local (RocksDB), sauvegardé dans un topic *changelog*.

- **Assurer la tolérance aux pannes**

Restauration depuis le *changelog*, et *standby replicas* pour aller plus vite.

SÉANCE 11

Au programme

01

Pourquoi l'état

la limite du stateless

02

Grouper & agréger

count, aggregate

03

State stores & changelogs

RocksDB, tolérance

04

Atelier

agréger par devise

01

- PARTIE 1

Pourquoi l'état

Certaines questions exigent de la mémoire

- Le stateless traite chaque enregistrement **isolément** : impossible de *compter* les commandes par devise ou de *cumuler* un chiffre d'affaires.
- Le traitement **avec état** conserve une valeur **par clé**, mise à jour à chaque enregistrement : un compteur, un cumul, un agrégat quelconque.

■ Stateless

`filter`, `mapValues` : aucune mémoire d'un message à l'autre.

Stateful

`count`, `aggregate` : un état par clé, qui évolue dans le temps.

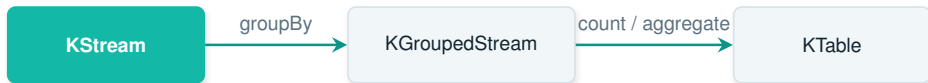
02

- PARTIE 2

Grouper & agréger

PARTIE 2 · LE PRINCIPE

Grouper, puis agréger



- `groupByKey` (clé inchangée) ou `groupBy` (nouvelle clé) produit un `KGroupedStream`.
- Une opération d'agrégation (`count`, `reduce`, `aggregate`) le transforme en `KTable` : un état courant par clé.

count et aggregate

Grouper par devise, puis compter et cumuler

```
KGroupedStream<String, String> byCcy =  
    orders.groupBy((k, v) -> currencyOf(v),  
        Grouped.with(Serdes.String(), Serdes.String()));  
  
byCcy.count(Materialized.as("count-by-ccy"));    // KTable<String,Long>  
  
byCcy.aggregate(() -> 0.0,  
    (ccy, v, sum) -> sum + amountOf(v),          // accumulateur  
    Materialized.with(Serdes.String(), Serdes.Double()));
```

aggregate = un *initialiseur* + un *agrégateur* (clé, valeur, accumulateur).

Voir l'agrégat évoluer

Lancer, produire, lire les totaux par devise

```
mvn -q compile exec:java \  
  -Dexec.mainClass=fr.esgi.kafka.tp11.CurrencyAggregator &  
echo 'o-1;19.9;eur' | kafka-console-producer.sh \  
  --bootstrap-server localhost:29092 --topic orders  
kafka-console-consumer.sh --bootstrap-server localhost:29092 \  
  --topic total-by-currency --property print.key=true
```

■ Par clé

La sortie est indexée par devise : EUR, USD, GBP...

■ Cumulatif

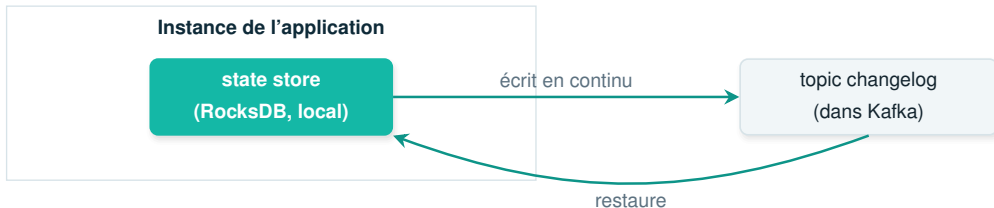
Chaque nouvelle commande met à jour le total de sa devise.

03

- PARTIE 3

State stores & changelogs

Un état local, sauvegardé dans Kafka



- Le `KTable` agrégé est **matérialisé** dans un store local (RocksDB par défaut, sous `state.dir`).
- Chaque mise à jour est aussi écrite dans un **topic changelog** compacté : `<application.id>-<store>-changelog`.

Reconstruire l'état après une panne

- Au **démarrage** ou après un **rééquilibrage**, l'instance qui reçoit une partition **rejoue son changelog** depuis le début pour reconstruire le store local.
- Pour de gros états, cette restauration prend du temps : les **standby replicas** (`num.standby.replicas`) maintiennent une copie à jour sur d'autres instances et accélèrent la reprise.

■ Restauration

L'état n'est jamais perdu : il se rejoue depuis le changelog.

■ Standby

Des copies tièdes réduisent le temps de bascule.

04

- PARTIE 4

Atelier — agréger par devise

Cluster et topics

Demarrer le cluster et creer les topics

```
docker compose up -d          # 3 brokers KRaft

for t in orders count-by-currency total-by-currency; do
  kafka-topics.sh --bootstrap-server localhost:29092 \
    --create --topic $t --partitions 3 --replication-factor 3
done
```

■ Projet

Maven + kafka-streams ;
replication.factor = 3.

■ Entrée

Lignes orderId;amount;currency
dans orders.

Construire l'agrégation

- 1 **Grouper** par devise : `groupBy + Grouped.with` (clés et valeurs String).
- 2 **Compter** : `count(Materialized.as(...))` → `count-by-currency`.
- 3 **Cumuler** : `aggregate` (somme des montants) → `total-by-currency`.
- 4 Lancer, produire des commandes, observer les agrégats par devise.
- 5 **Arrêter puis relancer** : l'état est restauré depuis le changelog.

Les topics internes et la reprise

Lire les agregats, puis lister les topics internes

```
kafka-console-consumer.sh --bootstrap-server localhost:29092 \  
  --topic total-by-currency --from-beginning --property print.key=true  
# Streams a cree des topics -repartition et -changelog  
kafka-topics.sh --bootstrap-server localhost:29092 --list
```

■ Internes

Un topic `-repartition` (groupBy) et des `-changelog` (stores).

■ Reprise

Après redémarrage, les totaux repartent de leur dernière valeur.

Pour aller plus loin

Objectif — varier les agrégations et durcir la tolérance aux pannes.

■ 1. reduce

Calculer le montant
maximum par devise
avec `reduce`.

■ 2. Exactly-once

Passer la garantie en
`exactly_once_v2`.

■ 3. En mémoire

Remplacer RocksDB par
un store *in-memory* et
comparer.

Au-delà de l'agrégation simple

- **Le fenêtrage** : agréger *par tranche de temps* (5 dernières minutes, par heure...), avec gestion des données en retard.
- **Les jointures** : enrichir un flux avec un autre flux ou une table (`KStream-KTable`, `KStream-KStream`).
- **Les Interactive Queries** : interroger directement le state store, sans repasser par un topic de sortie.

■ Acquis

Agréger un flux par clé, avec un état tolérant aux pannes.

À venir

Fenêtrer, joindre, puis exposer l'état (S12, S14).

SÉANCE 11

À retenir

- ✓ Le traitement **avec état** maintient une valeur **par clé**, mise à jour dans le temps.
- ✓ `groupBy/groupByKey` → `KGroupedStream`; `count/reduce/aggregate` → `KTable`.
- ✓ L'agrégat est **matérialisé** dans un state store local (RocksDB par défaut).
- ✓ Le store est sauvegardé dans un **changelog** :
`<application.id>-<store>-changelog`.
- ✓ Tolérance aux pannes : **restauration** depuis le changelog, **standby replicas** pour accélérer.



LA SUITE — SÉANCE 12

Fenêtrage & jointures

Agréger par tranche de temps et enrichir un flux avec un autre : KStream-KTable, fenêtres et grace period.